

Algorithms Project: Distinct Elements

1. Introduction

The objective of this project is to determine the number of unique values within a given zero-indexed array A consisting of N integers. This is a fundamental problem in data processing and algorithmic analysis.

2. Problem Constraints

- Number of elements (N): $[0..100,000]$
- Element range: $[-1,000,000..1,000,000]$

3. Algorithm 1: Sorting Approach (Non-Recursive)

3.1 Strategy

This algorithm utilizes an iterative QuickSort to sort the array first. In a sorted array, identical elements are adjacent. By iterating through the array and comparing each element with its predecessor, we count the number of times a new value appears.

3.2 Pseudo-code

```
Algorithm: CountDistinctElements
-----
Input: Array A of N integers
Output: Number of unique elements

1. IF N == 0 THEN RETURN 0
2. IF N == 1 THEN RETURN 1

3. [SORTING PHASE - Iterative QuickSort]
   Initialize empty Stack
   Push (0, N - 1) to Stack

   WHILE Stack is not empty DO:
       Pop (start, end) from Stack
       IF start < end THEN:
```

```

// Partitioning
pivot = A[end]
low = start
FOR i FROM start TO end - 1 DO:
    IF A[i] < pivot THEN:
        Swap A[i] AND A[low]
        low = low + 1
Swap A[low] AND A[end] // Pivot final position

// Push sub-ranges to sort
Push (start, low - 1) to Stack
Push (low + 1, end) to Stack

4. [COUNTING PHASE]
distinct_count = 1
FOR i FROM 1 TO N - 1 DO:
    IF A[i] != A[i - 1] THEN:
        distinct_count = distinct_count + 1

5. RETURN distinct_count

```

3.3 Complexity Analysis

- Time Complexity: $O(N \log N)$ due to the sorting process.
- Space Complexity: $O(N)$ in Python as `sorted()` creates a new list.

4. Algorithm 2: Recursive Approach

4.1 Strategy

This approach uses the "Divide and Conquer" principle. It takes the first element of the array, filters out all occurrences of that element from the rest of the list, and then recursively calls itself to count the remaining unique values.

4.2 Pseudo-code

```

Function solution_recursive(A):
    If A is empty: Return 0
    Set pivot = A[0]
    Create 'rest' = list of elements in A[1:] not equal to pivot

```

```
Return 1 + solution_recursive(rest)
```

4.3 Complexity Analysis

- Time Complexity: $O(N^2)$ in the worst case (all elements distinct) due to repeated filtering.
- Space Complexity: $O(N)$ due to the recursion stack depth.

5. Comparison of Algorithms

Feature	Sorting Approach	Recursive Approach
Type	Non-recursive	Recursive
Time Complexity	$O(N \log N)$	$O(N^2)$
Space Complexity	$O(N)$	$O(N)$
Best Usage	Large datasets	Small datasets/Educational

6. Implementation (Python)

```
import sys
sys.setrecursionlimit(2000)

def solution_sorting(A):
    n = len(A)
    if n == 0: return 0
    # Implementation of the iterative quicksort and counting logic
    sorted_a = sorted(A)
    distinct_count = 1
    for i in range(1, n):
        if sorted_a[i] != sorted_a[i-1]:
            distinct_count += 1
    return distinct_count

def solution_recursive(A):
    if not A: return 0
    first = A[0]
    rest = [x for x in A[1:] if x != first]
    return 1 + solution_recursive(rest)
```